CS 24302 : COMPILER DESIGN LABORATORY July 30 and 31, 2026

Lab Assignment 1 :

Objective :

- Learn the basic steps to work on a Linux server of the department – login credentials, create a new password, setting up a separate folder for the course, copy the entire folder for the lab session from the specified faculty folder into her/his designated folder for the session.
- Refresh the basic shell commands for files and directory resources in this OS.
- Get familiar with the working of a production quality open source compiler, gcc – the gnu C Compiler - various switches used for executing gcc and their effects. Identify the cousins of the compiler using the relevant switch that produces the required information.
- Write a simple C source code and experiment with gcc, using an appropriate switch to get a desired effect. Examine the attributes of the generated executable using relevant shell commands.
- Run the executable code with sample input to produce output using the file i/o feature of the shell.
- Find the running time of the executable program for a given sample input and output. Interpret the results generated by the "time" command. Find the average run time of the executable by repeating the above steps a few times.
- Use switches of gcc to generate internal dumps produced by gcc. Browse through a few selected dumps and observe their relationship with the given source. Get familiar with the naming schemes used by gcc for the dumped files and acquire an intuitive feel for the contents of a few dumps.
- Use gcc generate to assembly code for the given source and the students get a feel for the assembly code visa-vis the source.
- Use the optimization switch to generate better code by gcc. Time the optimized executable and compare the same with the unoptimized executable S
- Acquire ability to view and understand the internal dumps produced which shall be used as benchmark for the experiments that you will conduct henceforth.

Setting up for the Session :

Read the two files, "Lab-conduct-potocol.pdf" and "lab1-setting-up-handout.pdf" and follow the instructions provided there.

Problem 1 Consider the simple C program, "lab1prog.c" given to you for experimentation.

(a) Compile the C program using the command given below. Report all the files created by the execution of this command and determine the type of the generated file(s) with reasons.

```
$ gcc −c lab1prog.c
```

(b) Compile the given C program using the command given below and report the files created by the execution of this command and write the type of the generated file(s) with reasons.

```
$ gcc  lab1prog.c
```

(c) Compile the given C program using the command given below and report the files created by the execution of this command and write the type of the generated file(s) with reasons. Further write the difference between using 1(b) or 1(c) to compile and run the generated executable code.

```
$ gcc  lab1prog.c −o unopt
```

(d) Run the executable code generated by 1(b) and 1(c) by reading values from the file, "input.txt" and save the output in a file, named as "output1.txt" and output2.txt

```
$./a.out < input.txt > output1.txt
$./unopt < input.txt > output2.txt
```

Report your findings by examining and comparing the contents of the two output files.

Problem 2

(a) Compile the given C program using the command given below and report the files created by the execution of this command and explain the composition of the file, "output3.txt" with reasons.

```
$ gcc  -E lab1prog.c > output3.txt
```

(b)Find out all the files created by the execution of the above command. Open the file, "gcc-output.txt" and examine its contents. Determine and report your observations about, i) versions of the gcc and Linux/Unix, ii) number and names of the executables used, and iii) identify the names of as many cousins encountered in gcc as you can.

```
$ gcc  lab1prog.c −v > gcc-output.txt
```

Problem 3 (a) Determine the execution time of the following commands and write a report explaining the results displayed on screen for each command. Recommended to read the Appendix for options of gcc and for file i/o using the shell.

```
$time ./unopt < input.txt
$time ./unopt < input.txt
$time ./unopt < input.txt
```

(b) Run the following commands and write a report on the outputs generated. The switch O2 informs gcc to aggressively optimize the source C program.

```
$ gcc  lab1prog.c –O2 -o opt
$ time ./opt < input.txt
$ time ./opt < input.txt
$ time ./opt < input.txt
```

Report on the difference in run time performance with and without using optimization switch of gcc.

Problem 4. Generate the various intermediate dumps generated by the compiler for our given C source. Compare particularly the compiler dumps after the 5th and 6th passes named in ".original", and ".gimple" with the original C and report your findings.

$ gcc lab1prog.c

Problem 5. Generate annotated assembly codes for the given source, with and without using the switch "−O2". Compare the sizes and contents of the two assembly codes saved in the files, "lab1prog_unopt.s" and "lab1prog_opt.s" and report your observations about which appears to be more efficient.

```
$ gcc lab1prog.c –S –fverbose-asm
$ mv lab1prog.s lab1prog_unopt.s
$ gcc lab1prog.c –S –fverbose-asm –O2
$ mv lab1prog.s lab1prog_opt.s
```

APPENDIX : Basic Switches of gcc

1 **Use of the compiler switch −c** : this switch informs the compiler that the user is only interested in compiling the submitted program but does not want the executable to be generated.

2 **Use of the compiler switch −o** : this switch (letter o in small case) informs the compiler that the user is interested in giving her/his own name to the generated executable file and override the default name "a.out" used by the compiler. As a good practice, it is better to give a name to each executable program, instead of using the default name, because the latter gets overwritten after every compilation, while the former remains saved with us.

3 **Use of compiler switch −S** : assembly code is generated by the compiler and saved in the file "testprog.s". The assembly code generated directly by this switch is difficult to read.

$ gcc −S testprog.c will generate assembly code in testprog.s

A better human readable assembly code (annotated assembly) is generated using an additional switch, "−fverbose−asm" , for example $ gcc −S −fverbose−asm testprog.c will generate annotated assembly in testprog.s

4 Use of the verbose switch "−v" of gcc

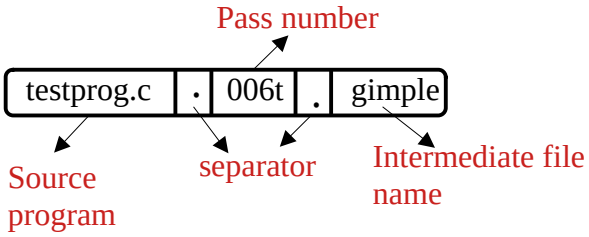
In contrast to the common misconception that gcc (gnu C compiler) is the compiler proper, it is in reality a shell script that calls various tools for performing compilation and related tasks. The switch "−v" can be used to know the internal structure of gcc. The switch "−v" usually stands for verbose description.

5 Generating intermediate compiler dumps in gcc

In order to generate the intermediate dumps produced by gcc, we use the switches given below. Note that there may be small variations in the dumps and their formats depending on

the gcc version used. The verbose dump gives the version of gcc and version of ubuntu that is used in my laptop. Earlier or later versions may give slightly different dumps, **gcc version 11.4.0 (Ubuntu 11.4.0-2ubuntu1~20.04).** Earlier or later versions would give slightly different dumps. $ gcc --version will inform the version of the compiler.

Observations : Let the command be $ gcc -fdump-tree-all testprog.c
The internal files that are generated are named as "labprog1.c.dddt.name", that is the source program name suffixed with dot (.) followed by 3 digits followed by t; which in turn is followed by a file-name. The naming convention used is the following.



6 Understanding the Intermediate Compiler Dumps

In the figure above, the file "gimple" is the dump produced by gcc after the 6th pass. Also it appears that gcc performed a large number of passes and the result are saved in an appropriately named intermediate dump file.

- It may be noted that gimple is the name of intermediate language used by gcc (other compilers may use AST, RTL, etc instead). The result of the 6th pass is therefore the source program translated into gimple language.

- The other passes use appropriate names to indicate the action performed by the pass. For example, the 5th pass gives the original program after processing; the 15th pass generates the control flow graph (cfg); the 23rd pass translated the intermediate code to static single assignment (ssa); 244th pass produces an optimized version, and so on.

File i/o using the Shell

A small digression for those who are NOT familiar with shell programming. There are 3 operators used by the shell for i/o redirection (redirecting the standard input, standard output and standard error files to other files chosen by an user rather than their default binding to device files.

Operator	Usage	Semantics
<	$ ./a.out < file1	The input required by a.out will be read from "file1"

		when the program reads using scanf() or equivalent functions.
> or 1>	$ ./a.out > file2 $ ./a.out 1> file2	The output produced by a.out, using printf() or equivalent, will be written to "file2"
2>	$ ./a.out 2> file3	The error messages produced by a.out, using the standard error channel, will be written to "file3"
>> or 2>>	$ ./a.out >> file4 $ ./a.out 2>> file5	Same effect as > or 2> except that the data is appended to the files instead of overwriting.

End of Laboratory Session 1